checkoutOs

Version 1.0 — Complete Engineering Handbook

Infrastructure-Grade Payment Orchestration

checkoutOs Engineering Team

Revision 1.1 — Includes Race Condition Fixes
April 14, 2026

Contents

Revision History

Version	Date	Changes
1.0	2024-01-15	Initial handbook release
1.1	2024-03-20	Added race condition fixes: webhook buffer parsing, order_id fallback lookup, polling SUCCESS blocking

Preface

This handbook is the single source of truth for the checkoutOs V1 payment orchestration system. It documents every architectural decision, layer contract, and operational requirement.

Critical

This system handles real money. The principles documented here are non-negotiable. Deviations from patterns described in this handbook will result in inconsistent payment states, failed refunds, or security vulnerabilities.

Part I

Project Overview & Standards

Chapter 1

Project Overview

1.1 What is checkoutOs?

Every payment gateway in India ships its own SDK, uses different naming conventions, exposes different status strings, and has unique integration quirks. Teams that integrate one provider get locked in — switching later requires rewriting core payment logic.

checkoutOs solves this with a single stable HTTP API that wraps native gateway SDKs behind a unified contract. Teams can switch providers by changing a single environment variable.

1.2 Core Principles

These principles are non-negotiable. Every design decision, code review, and architectural choice is evaluated against them.

Principle	What it means in practice
Correctness over convenience	Payment state must always be accurate — silent failures are unacceptable
Explicit over implicit	No magic, no hidden behaviour, no assumptions about caller intent
Contracts over assumptions	Every layer has a typed interface; every boundary is enforced
Guardrails over trust	Fail fast, validate everything at startup, throw on unknown states
Infrastructure-grade discipline	This system handles real money — treat it accordingly

1.3 V1.0 Scope

Version 1.0 is redirect-based and SDK-less.

Decision	Choice	Why
Checkout type	Redirect-based only	No PCI-DSS required
Integration style	SDK-less HTTP API	Any client, any language

State store	Redis	Persistent, multi-instance safe
HTTP client	axios	Typed, timeout-enforced, mock-server compatible

1.3.1 What Is In Scope

- Payment creation via unified API returning `paymentUrl`
- Status polling endpoint with staleness window
- Refund creation (full and partial)
- Webhook reception, signature verification, and relay
- Health check endpoint for load balancer integration
- Swagger/OpenAPI documentation
- Mock gateway server for local development

1.3.2 What Is Explicitly Out of Scope

Feature	Reason
API-based card capture	Requires PCI-DSS certification
Native mobile SDKs	Clients use HTTP API + WebView
UPI intent / deep links	Requires polling logic
Subscriptions / recurring billing	Requires mandates and separate flows
Webhook retry / DLQ	Deferred to V1.2
JWT / API key authentication	Deferred to V1.2
Auto gateway failover	Deferred to V1.2
Metrics and dashboards	Deferred to V1.2

1.4 Tech Stack

Component	Technology
Runtime	Node.js 18 (v20 recommended)
Language	TypeScript strict mode + `exactOptionalPropertyTypes`
Framework	Express.js
HTTP client	axios (gateway communication + webhook relay)
ID generation	UUID v4
Store	Redis v6+
Redis client	ioredis
Config validation	Zod
Logging	Winston
Mock server	MSW (@mswjs/http-middleware)
Containers	Docker + Compose
Documentation	OpenAPI + Swagger UI
Package manager	npm

1.5 Supported Gateways

Gateway	Status	Notes
Razorpay	V1.0 — fully implemented	Redirect-based, order → payment flow
PayU	V1.x — registry stub	Plugin pending
Cashfree	V1.x — registry stub	Plugin pending

Chapter 2

Engineering Standards

2.1 TypeScript Rules

checkoutOs uses `strict: true` with additional safety constraints.

Rule	Enforcement	Why
No implicit any	ESLint + TypeScript	Prevents hidden type bugs
Explicit return types on public functions	ESLint	Clear contracts at every boundary
exactOptionalPropertyTypes true	TypeScript	field?: string cannot be set to undefine
No unchecked indexed access	TypeScript	Prevents silent undefined at array/obje
No floating promises	ESLint	Silent async failures cause ghost payme
No default exports	ESLint	Explicit imports only — limited excepti
No any	ESLint	Forbidden throughout — tests slightly l

2.1.1 exactOptionalPropertyTypes Pattern

With this flag enabled, optional fields typed as `?: string` cannot be assigned `undefined` explicitly — the key must be omitted entirely when the value is absent.

```
// Wrong: assigns undefined explicitly, TypeScript error
return { description: raw.description };

// Correct: omits the key when undefined
return {
  amount: raw.amount,
  ...(raw.description != undefined && { description: raw.
      description })
};
```

Listing 2.1: Correct Pattern for exactOptionalPropertyTypes

2.2 ESLint, Prettier, and Git Hooks

- **ESLint**: Used as a correctness and safety tool. `any` is forbidden, floating promises are not allowed, explicit return types are required, default exports are disallowed.

- **Prettier**: Single source of truth for code formatting. Enforced automatically before every commit.
- **Git Hooks**: Husky manages hooks, lint-staged runs checks only on staged files.

2.3 Commit Discipline & Branch Naming

- Clear, descriptive commit messages. One logical change per commit. No direct commits to protected branches.
- **Branch Naming**: `feature/`, `fix/`, `refactor/`, `docs/`, `chore/`.

Part II

Build Steps Reference

Chapter 3

Complete Build Sequence

The full build sequence including all steps:

Step	Layer	Depends On
1	types	nothing
2	errors	types
3	utils	types
4	config	types, gateway registry
5	store	types, errors, config
6	gateways	types, errors, config
7	services	types, errors, config, store, gateways
8	controllers	types, errors, services, utils/asyncHandler
9	middleware	types, errors, utils/response
10	routes + app + server	controllers, middleware, gateway registry
11	tests	all of the above

3.1 Step 1: Types Layer

The foundation. Every other layer imports from here. Written first so contracts are established before any implementation.

Key types defined in `payment.types.ts`: `GatewayName`, `PaymentStatus` enum (8 states), `RefundStatus` enum, `CreatePaymentRequest`, `PaymentResponse`, `StoredPayment` (Redis shape), and `StoreRefund`.

3.2 Step 2: Errors Layer

All errors extend `AppError`. Never throw plain `Error` objects.

```
abstract class AppError extends Error {
  abstract readonly httpStatus: number;
  readonly code: ErrorCodeValue;
  readonly details: Record<string, unknown>;
  readonly isOperational: boolean; // true = client error, false =
      system failure
```

```
}
```

Listing 3.1: AppError Base Class

3.3 Step 3: Utils Layer

Pure functions. Zero dependencies on any other layer. No side effects. Exports: `generatePaymentId`, `generateRefundId`, `now()`, `toISOSTring()`, `success()`, `error()` response helpers, and a configured Winston logger.

3.4 Step 4: Config Layer

Responsibilities:

1. Read `process.env`
2. Validate against Zod schema with conditional gateway credential requirements
3. Export a single typed config object

3.5 Step 5: Store Layer

The only layer that interacts with Redis. Services, controllers, gateways, and middleware never access Redis directly. Exports functions for payment and refund Redis operations (CRUD, reverse lookups, status updates).

3.6 Step 6: Gateway Layer

Follows a registry-driven plugin architecture. The service layer calls `getActiveGateway()` — it never imports from a gateway folder directly. Each gateway is a self-contained folder with `*.types.ts`, `*.mapper.ts`, and `*.plugin.ts`.

Chapter 4

Step 7: Services Layer

4.1 What Services Do

Services own all business logic. They are the only layer that orchestrates between the gateway, store, config, and utils layers. Controllers call services. Services never call controllers.

4.1.1 Key Rules

- Services never import from `redis.client` directly — only store functions
- Services never import from any gateway folder directly — always via `getActiveGateway()`
- Services never read `process.env` — always via config
- All typed errors propagate upward — controllers do not catch and re-wrap them

4.2 health.service.ts

Single export: `checkHealth(): Promise<HealthResponse>`

Runs Redis (`pingRedis()`) and gateway (`plugin.healthCheck()`) checks concurrently via `Promise.allSettled`. Never throws — always returns a `HealthResponse`.

- `status: 'ok'` → both Redis and gateway are healthy
- `status: 'degraded'` → one of the two is healthy
- `status: 'error'` → both are unhealthy

4.3 payment.service.ts

4.3.1 createPayment(req: CreatePaymentRequest): Promise<PaymentResponse>

1. Validate amount — must be a positive integer in paise. Throws `InvalidAmountError` on failure.
2. Call `plugin.createPayment()` via `getActiveGateway()`.
3. Generate `chk_id` via `generatePaymentId()`.
4. Save `StoredPayment` to Redis: `gatewayOrderId` = gateway result ID (order_XXXX), `gatewayPaymentId` = "" (empty at creation — Option C), status = `PENDING`.
5. Return `PaymentResponse`.

4.3.2 Checkout URL Strategy (V1 Enhancement)

```
// The paymentUrl returned to the client is now internal, not
    gateway-hosted
const paymentUrl = `${config.app.baseUrl}/checkout/${chkId}`;
```

Listing 4.1: Checkout URL Generation

Note

Important: `paymentUrl` is derived, not persistent. It is NOT stored in Redis. This avoids duplication, prevents stale URLs, and maintains a single source of truth.

4.3.3 getPaymentStatus(chkId: string): Promise<PaymentStatusResponse>

Re-poll strategy for production:

- Terminal status (`SUCCESS`, `FAILED`, `REFUNDED`, etc.) $\rightarrow$ return Redis value immediately, never hits the gateway again.
- Non-terminal status (`PENDING`, `PROCESSING`) $\rightarrow$ check staleness (how long since `updatedAt`?).
- If ≤ 10 seconds (fresh): return Redis value.
- If > 10 seconds (stale): re-poll gateway, update Redis if changed.

Why 10 seconds? Razorpay's default rate limit is 300 requests/minute. Without the staleness window, every client poll would hit the gateway and exhaust the rate limit. With a 10-second window, at most 6 gateway calls go out per minute per payment.

4.3.4 Polling Logic — Race Condition Fix (V1.1)

Critical

Critical Change: Polling MUST NOT transition a payment to `SUCCESS`. Only the webhook sets `SUCCESS` to ensure `gatewayPaymentId` is always populated for terminal states.

```
export async function getPaymentStatus(chkId: string): Promise<
    PaymentStatusResponse> {
  const stored = await findPaymentByChkId(chkId);
  if (!stored) throw new PaymentNotFoundError(chkId);

  // Terminal states NEVER trigger polling
  if (isTerminalStatus(stored.status)) {
    return toPaymentStatusResponse(stored);
  }

  const ageMs = Date.now() - new Date(stored.updatedAt).getTime();
  if (ageMs < REPOLL_STALENESS_MS) {
    return toPaymentStatusResponse(stored);
  }

  const plugin = getActiveGateway();
```

```
16    const fresh = await plugin.getPaymentStatus(stored.gatewayOrderId
          );
17
18    // BLOCK 1: Never allow polling to transition to SUCCESS
19    // Webhook is the ONLY source of truth for SUCCESS
20    if (fresh.status === PaymentStatus.SUCCESS) {
21      logger.warn('Polling detected SUCCESS status - waiting for
            webhook backfill', {
22        chkId,
23        currentStatus: stored.status,
24        hasGatewayPaymentId: stored.gatewayPaymentId !== '',
25      });
26      return toPaymentStatusResponse(stored); // Return stale data
27    }
28
29    // BLOCK 2: Check for refund states requiring gatewayPaymentId
30    const requiresGatewayPaymentId = [
31      PaymentStatus.REFUNDED,
32      PaymentStatus.PARTIALLY_REFUNDED,
33    ].includes(fresh.status);
34
35    if (requiresGatewayPaymentId && stored.gatewayPaymentId === '') {
36      logger.warn('Polling detected refund status - waiting for
            webhook backfill', {
37        chkId,
38        freshStatus: fresh.status,
39      });
40      return toPaymentStatusResponse(stored);
41    }
42
43    // Only update safe transitions (e.g., PENDING -> PROCESSING)
44    if (fresh.status !== stored.status) {
45      await updatePaymentStatus(chkId, fresh.status);
46      stored.status = fresh.status;
47      stored.updatedAt = new Date().toISOString();
48    }
49
50    return toPaymentStatusResponse(stored);
51  }
```

Listing 4.2: Polling Logic with SUCCESS Block

4.3.5 validateAndPrepareRefund(chkId, req)

Exported for use by `RefundService`. Validates in order:

1. Payment exists in Redis (throws `PaymentNotFoundError`).
2. Status is `SUCCESS` or `PARTIALLY_REFUNDED` (throws `RefundNotAllowedError` otherwise).
3. Refund amount is a positive integer in paise (defaults to full amount if omitted).
4. Refund amount does not exceed remaining refundable (reads `findRefundsByChkId()` to sum existing refunds; throws `RefundAmountExceedsPaymentError`).
5. `gatewayPaymentId` is not empty (throws `RefundNotAllowedError` with 'AWAITING_WEBHOOK' message).

4.4 refund.service.ts

4.4.1 createRefund(chkId, req)

1. Call `validateAndPrepareRefund()` from `payment.service`.
2. Call `plugin.createRefund(gatewayPaymentId: pay_XXXX, amount)`.
3. Generate `ref_id` via `generateRefundId()`.
4. Save `StoreRefund` to Redis.
5. Update parent payment status: `REFUNDED` if full amount, else `PARTIALLY_REFUNDED`.
6. Return `RefundResponse`.

4.4.2 getRefundStatus(refId: string)

Same staleness strategy as `getPaymentStatus`.

4.5 webhook.service.ts

Single export: `processWebhook(gateway, body, headers): Promise<void>`

4.5.1 V1 Philosophy: Webhook as Source of Truth

Webhook > Polling

4.5.2 Webhook Processing — Complete Flow (V1.1)

```
export async function processWebhook(
  gateway: GatewayName,
  body: unknown,
  headers: Record<string, string | string[] | undefined>
): Promise<void> {
  const plugin = getActiveGateway();

  // Step 1: Parse and verify webhook
  const event = plugin.parseWebhookEvent(body, headers);

  // Step 2: Try primary lookup by gatewayPaymentId
  let chkId = await findChkIdByGatewayId(gateway, event.
      gatewayPaymentId);

  // Step 3: FALLBACK - Try order_id from raw webhook payload
  // This is critical for the FIRST webhook where gatewayPaymentId
  // doesn't exist in our reverse lookup yet
  if (chkId === null) {
    let orderId: string | undefined;
    try {
      const rawBody = typeof body === 'string' ? body :
                      Buffer.isBuffer(body) ? body.toString('utf-8'
                          ) :
                      JSON.stringify(body);
      const rawWebhook = JSON.parse(rawBody) as Record<string,
          unknown>;
      orderId = (rawWebhook as { payload?: { payment?: { entity?: {
          order_id?: string } } } })
```

```
                ?.payload?.payment?.entity?.order_id;
    } catch {
      // Ignore parsing errors
    }

    if (orderId) {
      chkId = await findChkIdByGatewayId(gateway, orderId);
      if (chkId) {
        logger.info('Found payment via order_id fallback', {
            orderId, chkId });
      }
    }
  }

  if (chkId === null) {
    logger.warn('Webhook received for unknown payment', {
      gateway,
      gatewayPaymentId: event.gatewayPaymentId,
      reason: 'no_chk_id_found',
    });
    return; // NEVER throw here - would cause gateway to retry
  }

  const stored = await findPaymentByChkId(chkId);
  if (!stored) {
    logger.error('Payment not found after lookup', { chkId });
    return;
  }

  let paymentUpdated = false;

  // Step 4: Backfill gatewayPaymentId FIRST (before status check)
  // This ensures Option C compliance: gatewayPaymentId is
     populated
  // even if the status hasn't changed
  if (stored.gatewayPaymentId === '' && event.gatewayPaymentId !==
     '') {
    await updateGatewayPaymentId(chkId, gateway, event.
       gatewayPaymentId);
    paymentUpdated = true;
    logger.info('Gateway payment ID set from webhook', {
      chkId,
      gatewayPaymentId: event.gatewayPaymentId,
    });
  }

  // Step 5: Idempotent Status Update
  if (event.status !== stored.status) {
    await updatePaymentStatus(chkId, event.status);
    paymentUpdated = true;
    logger.info('Payment status updated from webhook', {
      chkId,
      oldStatus: stored.status,
      newStatus: event.status,
    });
  }
```

```
77
78    if (!paymentUpdated) {
79      logger.debug('Webhook received but no changes needed', { chkId
          });
80    }
81
82    // Step 6: Relay webhook (fire and forget)
83    await relayWebhook(chkId, event);
84  }
```

Listing 4.3: Webhook Processing with Fallback Lookup

4.5.3 Critical Rationale

- **Fallback Lookup:** The first webhook contains `pay_XXX` which doesn't exist in our reverse lookup yet. We extract `order_id` from the raw payload to find the payment. This fixes the "unknown payment" race condition.
- **Backfill First:** `gatewayPaymentId` is set before checking status equality. This guarantees it's populated for refunds even if the status was already updated by polling.
- **No Throw on Unknown:** If relay fails and the webhook handler returns non-200, the gateway retries the webhook, leading to duplicate state updates. Internal failures are logged and monitored. The gateway always gets a 200 after signature verification.
- **Update Before Relay:** Redis is always updated before relay is attempted. If relay fails, state remains consistent.

Chapter 5

Step 8: Controllers Layer

5.1 What Controllers Do

Controllers have exactly one job: parse the HTTP request, call the service, send the response. No business logic lives here. No try/catch blocks are needed.

5.1.1 asyncHandler Utility

Located at `src/utils/asyncHandler.ts`. Wraps async Express handler functions and forwards any rejection to `next(err)`.

```
// Without asyncHandler - boilerplate
export async function createPayment(req, res, next) {
  try {
    const result = await paymentService.createPayment(req.body);
    res.status(201).json(success(result));
  } catch (err) {
    next(err);
  }
}

// With asyncHandler - happy path only
export const createPayment = asyncHandler(async (req, res) => {
  const result = await paymentService.createPayment(req.body);
  res.status(201).json(success(result));
});
```

Listing 5.1: asyncHandler Usage

5.1.2 Why Named Function Exports?

Every other layer in checkoutOs uses named exports. Controller classes require instantiation and `.bind()` calls in routes — missing a `.bind()` is a silent runtime bug. Named exports are just functions.

5.1.3 Why All Errors Go to next(err)?

The `AppError` base class already carries `httpStatus`, `code`, etc. The error middleware formats the correct HTTP response from those fields. One error handler, one mapping location.

5.2 Controller Implementations

5.2.1 health.controller.ts

```
export const check = asyncHandler(async (_req, res) => {
  const health = await checkHealth();
  const httpStatus = health.status === 'ok' ? 200 : 503;
  res.status(httpStatus).json(success(health));
});
```

Why 503 for degraded/error health? Load balancers read the HTTP status code to determine if a node should receive traffic. 503 removes unhealthy nodes automatically.

5.2.2 payment.controller.ts

Three handlers: `create` (POST /payments → 201), `getStatus` (GET /payments/:chkId → 200), `refund` (POST /payments/:chkId/refund → 201).

5.2.3 webhook.controller.ts

Critical

Critical body parsing note: The route uses `express.raw({ type: 'application/json' })` instead of `express.json()`. The raw bytes are needed to compute the HMAC-SHA256 signature. If `express.json()` runs first, it consumes the body stream — the original byte order is lost, so the HMAC never matches.

```
export const receive = asyncHandler(async (req, res) => {
  const gateway = req.params.gateway as GatewayName;
  const body: unknown = req.body; // Buffer, not parsed object
  const headers = req.headers;
  await processWebhook(gateway, body, headers);
  res.status(200).json({ received: true }); // always 200 after
      signature passes
});
```

5.2.4 checkout.controller.ts — V1 Addition

```
export const renderCheckout = asyncHandler(async (req, res) => {
  const { chkId } = req.params;
  const data = await paymentService.getPaymentStatus(chkId);

  // Terminal state handling
```

```
  if (data.status === 'SUCCESS') {
    return res.send(renderSuccessPage(data));
  }
  if (data.status === 'FAILED') {
    return res.send(renderFailurePage(data));
  }

  return res.send(renderCheckoutPage(data));
});
```

Listing 5.2: Checkout Controller with Terminal State Handling

CheckoutViewData must include:

- chkId
- status
- gatewayOrderId
- amount
- currency
- gateway

5.2.5 Razorpay Handler Fix (Critical V1 Change)

```
// Previous (broken): /checkout/success?chkId=...
// Problem: Route mismatch -> interpreted as chkId = "success"

// New (correct):
handler: function () {
  window.location.href = `/checkout/${chkId}`;
}
// Result: Single unified route, controller decides final state
```

Chapter 6

Step 9: Middleware Layer

6.1 error.middleware.ts

The single place where errors become HTTP responses. All controllers use `asyncHandler` which forwards errors to `next(err)`. This middleware catches all of them.

6.1.1 Three cases handled:

1. **AppError instance:** Use `err.httpStatus`, `err.code`, `err.message`, `err.details`. `isOperational` drives log level (true=warn, false=error).
2. **Unknown error** (plain Error, thrown string): Always return 500 `INTERNAL_ERROR`. Never expose internal message to client.
3. **Headers already sent:** Return early — do not write a second response.

Must be the last middleware registered in `app.ts`.

6.2 not-found.middleware.ts

Registered after all routes. Returns `ApiErrorResponse` with `NOT_FOUND` code for any unmatched route. Returns the response directly (does not call `next(err)`) because a 404 is not a system error.

Chapter 7

Step 10: Routes, App, and Server

7.1 Route Files

Each route file is intentionally minimal — 4 to 8 lines. Routes have one job: declare which HTTP method and path maps to which controller function.

```
export const paymentRouter = Router();
paymentRouter.post('/', create);
paymentRouter.get('/:chkId', getStatus);
paymentRouter.post('/:chkId/refund', refund);
```

Listing 7.1: payment.routes.ts

7.2 app.ts — Middleware and Mount Order

Order matters. Middleware forms a pipeline — each layer runs in the order it was registered.

```
app.use(requestLogger);       // 1st: logs every request
app.use('/webhooks', webhookRouter); // 2nd: BEFORE express.json()
    - raw body
app.use(express.json());      // 3rd: parses JSON for all other
    routes
app.use('/payments', paymentRouter);
app.use('/refunds', refundRouter);
app.use('/health', healthRouter);
app.use('/', checkoutRouter); // V1: checkout routes

// after all routes
app.use(notFoundHandler);     // absolute last
app.use(errorHandler);
```

Listing 7.2: app.ts Middleware Order

Critical

Why webhook router before express.json()? `express.json()` consumes the body stream. Mounting the webhook router first ensures `express.raw()` is the first to touch the body stream for `/webhooks/*` requests.

7.2.1 buildApp() and initialise()

`app.ts` exports two functions:

1. `buildApp(): Application` — Creates Express app with all middleware and routes. Does NOT call `app.listen()`. Makes the app testable without binding to a port.
2. `initialise(): Promise<void>` — Registers gateway plugin, connects to Redis. Called by `server.ts` before `listen()`. Any failure exits the process.

7.3 server.ts — Entry Point and Graceful Shutdown

Three responsibilities:

1. Call `initialise()` — gateway + Redis ready before traffic.
2. Call `buildApp()` — create Express application.
3. Call `app.listen()` — start accepting connections.

7.3.1 Graceful Shutdown

Handles `SIGTERM` and `SIGINT`:

- `server.close()` — stop new connections, finish in-flight requests.
- `disconnectRedis()` — close Redis connection cleanly.
- Safety timeout of 10 seconds — if shutdown takes longer, force `process.exit(1)`.

Chapter 8

Step 11: Testing

8.1 Test Framework and Structure

Framework: Vitest (native TypeScript, fast). Directory structure:

```
tests/
  unit/ (utils/, gateway/, middleware/, services/)
  integration/ (store/, api/)
  helpers/ (app.helper.ts)
vitest.config.ts (two test projects: unit + integration)
vitest.setup.unit.ts (ioredis mock, logger mock)
vitest.setup.integration.ts (real Redis)
```

8.2 Two Test Projects

- **Unit tests:** No real infrastructure. ioredis replaced with `ioredis-mock`. Logger mocked. 5-second timeout. Run with `npm run test:unit`.
- **Integration tests:** Real Redis required. Real Express app via supertest. Gateway HTTP calls intercepted via axios mock. Files run serially (share one Redis instance). Run with `npm run test:integration`.

8.3 Critical Patterns

8.3.1 Environment Variables Before Imports

Environment variables must be set at the top level of setup files — not inside `beforeAll()`:

```
// WRONG: too late, config has already parsed process.env
beforeAll(() => {
  process.env['ACTIVE_GATEWAY'] = 'razorpay';
});

// CORRECT: at the top of the setup file
process.env['ACTIVE_GATEWAY'] = 'razorpay';
import { vi, beforeAll } from 'vitest';
```

8.3.2 vi.mock() Hoisting

Use `vi.hoisted()` for variables referenced inside a `vi.mock()` factory:

```
const { mockPost } = vi.hoisted(() => ({ mockPost: vi.fn() }));
vi.mock('axios', () => ({
  default: {
    create: vi.fn().mockReturnValue({ post: mockPost })
  }
}));
```

8.3.3 Gateway Registry Mock

When mocking `gateway.registry`, the mock factory must re-export all constants that `config/index.ts` imports at module load time (e.g., `supportedGateways`, `gatewayEnvDefinitions`).

8.4 Test Priority Order

1. `razorpay.mapper.test.ts` — Every status mapping correct; unknown statuses always throw.
2. `error.middleware.test.ts` — AppError → HTTP mapping; internal errors never exposed.
3. `payment.service.test.ts` — Amount validation; Option C; staleness window; SUCCESS blocking.
4. `webhook.service.test.ts` — Unknown payments silent; gatewayPaymentId update; order_id fallback; relay fire-and-forget.
5. `payment.store.test.ts` — Option C atomic write of hash field and reverse lookup.
6. `refund.store.test.ts` — Multi-refund tracking.
7. `webhook.api.test.ts` — Raw body preserved; 200 for unknown payments; 401 for bad signature.
8. `payment.api.test.ts` — Full HTTP → service → Redis chain.
9. `id.test.ts`, `time.test.ts` — ID format, timestamp correctness.

Part III

Architecture, Operations & Integration

Chapter 9

Architecture & Design

9.1 How a Payment Works — V1 Enhanced Flow

9.1.1 High-Level Flow (V1)

Client POST /payments → checkoutOs creates payment on gateway → Returns {`paymentId`, `paymentUrl`: "/checkout/{chkId}", `status`} → Client opens checkout page → Gateway SDK handles payment internally → Redirect back to `/checkout/{chkId}` → Controller renders success/failure/retry → Gateway fires webhook → checkoutOs normalizes and relays webhook.

9.1.2 Step-by-Step Breakdown (V1)

Step	Actor	Description
1	Client → checkoutOs	POST /payments with amount, currency, orderId
2	checkoutOs → Gateway	Creates payment; stores gatewayOrderId in Redis
3	checkoutOs → Client	Returns paymentId (chk_), paymentUrl: "/checkout/{chkId}", PENDING
4	Client → Browser	Opens /checkout/{chkId} (internal page)
5	Browser → Gateway SDK	Razorpay/PayU SDK loads and handles payment UI
6	Gateway → Browser	User completes payment on gateway-hosted modal
7	Browser → checkoutOs	Redirect to /checkout/{chkId} (same route)
8	checkoutOs Controller	Reads status from Redis; renders success/failure/retry page
9	Gateway → checkoutOs	Fires payment.captured webhook with pay_XXXX
10	checkoutOs → Store	Updates gatewayPaymentId, status to SUCCESS
11	checkoutOs → Developer	Relays normalized webhook to WEBHOOK_RELAY_URL

9.2 Layer Architecture

Layer	File location	Responsibility
Controllers	`src/controllers/`	HTTP request parsing and response sending only
Services	`src/services/`	Business logic and orchestration

Gateways	`src/gateways/`	External gateway HTTP calls + response translation
Store	`src/store/`	Redis persistence — only layer that touches Redis
Middleware	`src/middleware/`	Cross-cutting concerns: logging, errors, body parsing
Types	`src/types/`	Shared TypeScript interfaces — no runtime code
Errors	`src/errors/`	Typed error classes — all extend AppError
Utils	`src/utils/`	Pure helper functions — zero side effects
Config	`src/config/`	Environment validation and config export

9.3 Unified Payment Status

Status	Meaning	Terminal?
PENDING	Payment initiated	No
PROCESSING	Processing in progress	No
SUCCESS	Payment completed	Yes
FAILED	Payment failed	Yes
REFUNDED	Full refund issued	Yes
PARTIALLY_REFUNDED	Partial refund issued	Yes
CANCELLED	User cancelled	Yes
EXPIRED	Session expired	Yes

9.4 Gateway ID Management (Option C)

Moment	`gatewayOrderId`	`gatewayPaymentId`
`createPayment()` **returns**	`order_XXX` (known)	`""` (empty)
`payment.captured` **webhook**	unchanged	`pay_XXX` (set by WebhookService)
`createRefund()` **called**	not used	`pay_XXX` passed directly

9.5 API Endpoints (V1 Updated)

Method	Path	Description
POST	`/payments`	Create a new payment
GET	`/payments/:chkId`	Get payment status
POST	`/payments/:chkId/refund`	Create a refund
GET	`/refunds/:refId`	Get refund status
GET	`/checkout/:chkId`	Render checkout page (V1)
POST	`/webhooks/:gateway`	Receive gateway webhook
GET	`/health`	Service health check
GET	`/docs`	Swagger UI

9.6 Redis Schema

9.6.1 Key Patterns

- `chk:pay:{chk_id}` — Hash full StoredPayment record
- `chk:gw:{gateway}:{gw_id}` — String reverse lookup: `gw_id` → `chk_id`
- `chk:ref:{ref_id}` — Hash full StoreRefund record
- `chk:ref:by-pay:{chk_id}` — Set all ref_ids for a payment

9.6.2 StoredPayment Hash Fields

Field Notes	Type	Set at
`chkId` **`chk_` prefixed internal ID**	string	creation
`gatewayOrderId` **`order_XXXX` — used for status polling**	string	creation
`gatewayPaymentId` **`pay_XXXX` — used for refunds; empty until set**	string	after webhook
`gateway` **Active gateway name**	string	creation
`orderId` **Developer's own order reference**	string	creation
`amount` **Stored as string in Redis hash**	string (paise)	creation
`currency` **INR in V1.0**	string	creation
`status` **PaymentStatus enum value**	string	creation + updates
`createdAt` **ISO 8601**	string	creation
`updatedAt` **ISO 8601**	string	every update

Note

V1 Note: `paymentUrl` is NOT stored in Redis — it is derived from `chkId` and `config.app.baseUrl`.

9.7 Race Condition Resolution — Complete Flow

9.7.1 Payment Creation Flow

```
POST /payments
    |
    v
payment.service.createPayment()
    |
```

```
    v
razorpay.plugin.createPayment() -> order_XXX
    |
    v
savePayment() -> Redis Hash + reverse lookup (order_XXX -> chkId)
    |
    v
Return { paymentId, paymentUrl: /checkout/:chkId }
```

9.7.2 Webhook Processing Flow (Updated V1.1)

```
POST /webhooks/razorpay
    |
    v
webhook.controller.receive()
    |
    v
razorpay.plugin.parseWebhookEvent() -> parses Buffer, verifies HMAC signature
    |
    v
webhook.service.processWebhook()
    |
    +-- Try lookup by pay_XXX
    |
    +-- FALLBACK: extract order_id from raw payload, lookup by order_XXX
    |
    +-- Backfill gatewayPaymentId (CRITICAL: before status check)
    |
    +-- Update status if changed
    |
    +-- Relay to developer (fire-and-forget)
```

9.7.3 Status Polling Flow (Updated V1.1)

```
GET /payments/:chkId
    |
    v
payment.service.getPaymentStatus()
    |
    +-- Terminal status? -> return Redis (no gateway call)
    |
    +-- Fresh (< 10s)? -> return Redis (rate limit protection)
    |
    +-- Poll gateway
    |
    +-- BLOCK: SUCCESS detected? -> return stale Redis (wait for webhook)
    |
    +-- BLOCK: Refund state without pay_XXX? -> return stale Redis
    |
    +-- Update safe transitions only (PENDING -> PROCESSING)
```

Chapter 10

Gateway Integration Guide

10.1 Overview

Every gateway is a self-contained folder under `src/gateways/`. The rest of the system never imports from a gateway folder directly.

10.2 File Structure Per Gateway

```
src/gateways/<gateway>/
  <gateway>.types.ts   // Raw API response shapes
  <gateway>.mapper.ts  // Implements GatewayMapper
  <gateway>.plugin.ts  // Implements GatewayPlugin
```

10.3 Integration Steps

1. **Define Raw Types** (`<gateway>.types.ts`) — TypeScript interfaces for raw API responses.
2. **Implement the Mapper** (`<gateway>.mapper.ts`) — Pure translation layer. Zero dependencies on axios, config, Redis, or errors.
3. **Implement the Plugin** (`<gateway>.plugin.ts`) — Owns all HTTP communication and webhook signature verification.
4. **Register the Gateway** — Add to `gateway.registry.ts`, `env.schema.ts`, and `.env.example`.

10.4 Gateway Plugin Implementation — Webhook Parsing

```
1 public parseWebhookEvent(body: unknown, headers: ...): WebhookEvent
      {
2   // Step 1: Signature verification
3   const signature = extractHeader(headers, 'x-razorpay-signature');
4   if (!signature) {
5     throw new GatewayInvalidSignatureError(this.name);
6   }
7
8   const rawBody = serializeBody(body);
9   const expected = crypto
```

```
    .createHmac('sha256', this.creds.webhookSecret)
    .update(rawBody)
    .digest('hex');

  // Constant-time comparison
  const sigBuffer = Buffer.from(signature, 'hex');
  const expBuffer = Buffer.from(expected, 'hex');
  if (sigBuffer.length !== expBuffer.length ||
      !crypto.timingSafeEqual(sigBuffer, expBuffer)) {
    throw new GatewayInvalidSignatureError(this.name);
  }

  // Step 2: Parse payload
  let webhook: RazorpayWebhookPayload;
  try {
    webhook = JSON.parse(rawBody) as RazorpayWebhookPayload;
  } catch {
    throw new GatewayMappingError('Invalid JSON in webhook body',
        this.name);
  }

  // Step 3: Extract and validate
  const { event } = webhook;
  const paymentEntity = webhook?.payload?.payment?.entity;
  if (!paymentEntity) {
    throw new GatewayMappingError(
      `Missing payment.entity for event: ${event}`,
      this.name
    );
  }

  // Step 4: Extract data
  const gatewayPaymentId = paymentEntity.id ?? '';
  const status = this.mapWebhookEventToStatus(event, paymentEntity)
      ;
  const amount = paymentEntity.amount ?? 0;
  const currency = paymentEntity.currency ?? 'INR';

  return {
    gateway: this.name,
    gatewayPaymentId,
    event,
    status,
    amount,
    currency: currency as 'INR',
    raw: body,
  };
}
```

Listing 10.1: Razorpay Plugin parseWebhookEvent (V1.1)

10.5 Testing Checklist for New Gateway

- `toUnifiedStatus()` throws `GatewayMappingError` for any unrecognized status string.

- `toUnifiedRefundStatus()` throws for any unrecognized refund status.
- `parseWebhookEvent()` throws `GatewayInvalidSignatureError` on invalid or missing signature.
- `parseWebhookEvent()` correctly handles Buffer bodies (doesn't assume parsed object).
- `createRefund()` makes exactly one API call — no internal ID resolution.
- `healthCheck()` never throws — returns `{ healthy: false }` on any failure.
- All HTTP calls throw `GatewayTimeoutError` on timeout, `GatewayUnavailableError` on other errors.
- Plugin compiles with `strict: true` and `exactOptionalPropertyTypes: true`.
- No `any` types anywhere — ESLint passes cleanly.
- Mock server handlers cover: payment creation, status check, refund, webhook.
- No circular dependencies (e.g., no logger import that depends on config that depends on plugin).

Chapter 11

Operations & Deployment

11.1 Startup Validation

checkoutOs will not start with missing or invalid configuration. The startup sequence must complete before traffic is accepted. `GET /health` is the liveness probe.

11.2 Health Endpoint

```
GET /health
{
  "status": "ok" | "degraded" | "error",
  "timestamp": "2024-01-15T10:30:00.000Z",
  "services": {
    "redis": { "healthy": true, "latencyMs": 1 },
    "gateway": { "healthy": true, "latencyMs": 45 }
  }
}
```

Listing 11.1: Health Response

11.3 Webhook Configuration

Configure your gateway's dashboard to fire webhooks to:

`POST https://your-checkouts-host/webhooks/razorpay`

11.4 Relay Failure Handling

If relay to `WEBHOOK_RELAY_URL` fails, checkoutOs logs a structured warning and returns 200 to the gateway. The payment state in Redis is always updated before relay is attempted.

Note

Why return 200 on relay failure? If relay fails and we return non-200, the gateway retries the webhook. This would cause duplicate state updates (idempotent, but wasteful) and potential race conditions. The relay is fire-and-forget. Internal monitoring should alert on relay failure logs.

11.5 Gateway Rate Limits

The staleness window in `getPaymentStatus` (10 seconds) is calibrated for Razorpay's default limit of 300 requests/minute. Terminal statuses never generate gateway calls.

Chapter 12

Common Mistakes Reference

12.1 Services Layer

Mistake	Consequence	Fix
Importing from `redis.client` directly	Bypasses store layer abstraction	Import from `store/payment.store` or `store/refund.store` only
Importing from a gateway folder	Bypasses registry	Always use `getActiveGateway()`
Reading `process.env`	Bypasses config validation	Use `config.webhook.relayUrl` etc.

12.2 Controllers Layer

Mistake	Consequence	Fix
Adding business logic	Violates layer contract	Move to service layer
Using try/catch instead of asyncHandler	Boilerplate that does nothing	Wrap with `asyncHandler`
Class-based controller	Requires `.bind()` in routes, silent bugs	Named function exports only

12.3 Testing

Mistake	Consequence	Fix
Setting env vars in `beforeAll()`	Config already parsed, vars ignored	Set at top level of setu
Referencing variables inside `vi.mock()` factory	Hoisting error — variable is undefined	Use `vi.hoisted()`

Missing exports in gateway registry mock	Config validation fails during test setup	Include `supportedGat` `gatewayEnvDefinitio` etc.
Running integration tests without Redis	All store tests fail	`docker start` `checkoutos-redis` firs

12.4 Webhook Flow

Mistake	Consequence	Fix
Throwing on unknown payment	Gateway retry storm	Log warning, return silently
Returning non-200 after relay failure	Gateway retries, duplicate state updates	Relay is fire-and-forget — a ways return 200
Updating Redis after relay	Relay failure leaves state inconsistent	Always update Redis BEFOR relay
Removing the `express.raw()` middleware	Signature verification always fails	Webhook route must use ra body parser
Not implementing `order_id` fallback	First webhook can't find payment	Parse raw payload f `order_id` and use as fa back lookup
Checking status before backfill	`gatewayPaymentId` remains empty for unchanged status	Backfill `gatewayPaymentI` BEFORE status equality che

Appendix A

Running the Application

A.1 Starting Everything

1. Redis: `docker start checkoutos-redis`
2. Mock gateway server: `npm run mock:gateway` (listens on `http://localhost:9090`)
3. checkoutOs app: `npm run dev` (listens on port 3000)

A.2 Example .env for Local Development with Mock Server

```
NODE_ENV=development
PORT=3000
ACTIVE_GATEWAY=razorpay
REDIS_URL=redis://localhost:6379
WEBHOOK_RELAY_URL=http://localhost:4000/webhook
RAZORPAY_BASE_URL=http://localhost:9090
RAZORPAY_KEY_ID=rzp_test_mockKeyId0001
RAZORPAY_KEY_SECRET=mockSecret00001
RAZORPAY_WEBHOOK_SECRET=mockWebhookSecret001
APP_BASE_URL=http://localhost:3000
```

A.3 Verifying Everything Works

```
# 1. Health check
curl -s http://localhost:3000/health | jq

# 2. Create a payment
curl -s -X POST http://localhost:3000/payments \
  -H "Content-Type: application/json" \
  -d '{"amount":50000,"currency":"INR","orderId":"test_001"}' | jq

# 3. Check status (copy paymentId from step 2)
curl -s http://localhost:3000/payments/chk_YOUR_ID | jq

# 4. Open checkout page (V1)
# Visit http://localhost:3000/checkout/chk_YOUR_ID in browser
```

Listing A.1: Verification Commands

Appendix B

V1 Enhancements Summary

1. Client calls `POST /payments`
2. Receives `/checkout/{chkId}`
3. Opens checkout page
4. Gateway SDK (Razorpay) handles payment in modal
5. Redirect back to `/checkout/{chkId}`
6. Controller renders success / failure / retry based on Redis state
7. Webhook updates final state asynchronously

Key Design Decisions (V1)

- **No gateway URL exposure** — internal redirect only
- **Redis = source of truth**
- **Webhook-first architecture** — webhook updates state, polling is fallback
- **Idempotent webhook processing** — safe for retries
- **Gateway Payment ID backfill** — populated on first webhook
- **Derived paymentUrl** — not stored, computed from `baseUrl` and `chkId`
- **Terminal state handling in checkout controller** — prevents post-payment errors
- **Single unified checkout route** — `/checkout/:chkId` handles all states

V1.1 Race Condition Fixes

- **Webhook Buffer Parsing:** `parseWebhookEvent` properly handles Buffer bodies instead of assuming parsed objects.
- **Order ID Fallback:** When `gatewayPaymentId` lookup fails, extract `order_id` from raw webhook payload and use as fallback.
- **Backfill Reordering:** Set `gatewayPaymentId` BEFORE checking status equality to ensure it's populated for refunds.
- **Polling SUCCESS Block:** Prevent polling from transitioning to `SUCCESS` — only webhook sets `SUCCESS`.
- **Polling Refund Block:** Prevent polling from transitioning to refund states without `gatewayPaymentId`.
- **Circular Dependency Removal:** Removed logger import from plugin to eliminate circular dependency.

Appendix C

Testing Verification Logs

C.1 Successful Test Output (V1.1)

```
[info] [payment-service] Payment created {"chkId":"chk_...","gatewayOrderId":"order_..."}
[info] [webhook-service] Found payment via order_id fallback {"orderId":"order_...","chkI
[info] [webhook-service] Gateway payment ID set from webhook {"gatewayPaymentId":"pay_...
[info] [webhook-service] Payment status updated from webhook {"oldStatus":"PENDING","newS
[debug] [payment-service] Payment status terminal - serving from Redis {"status":"SUCCESS
```

C.2 Redis State After Success

```
# Payment hash
HGETALL chk:pay:chk_xxx
  status: SUCCESS
  gatewayPaymentId: pay_xxx

# Reverse lookups
GET chk:gw:razorpay:order_xxx -> chk_xxx
GET chk:gw:razorpay:pay_xxx -> chk_xxx
```